Attention Is All You Need

NIPS 2017（现称 NeurIPS）· 2017 年 · Ashish Vaswani 等

摘要

机器翻译需要把源语言序列转换成目标语言序列。论文针对循环网络训练时的逐位置依赖、以及远距离信息传递路径较长的问题，提出 Transformer：用多头注意力交换上下文信息，配合前馈网络与位置编码完成编码和解码。原文表 2 在 WMT 2014 的 newstest2014 测试集上报告：英德翻译 base 为 27.3、big 为 28.4 BLEU，对照 ConvS2S 为 25.16；英法翻译 big 为 41.8，ConvS2S 为 40.46 BLEU。BLEU 越高越好。

符号表

符号	中文含义	类型 / 维度	作用	出处 / 适用范围
Q	查询矩阵	$n \times d_k$	决定当前位置要查询什么	§3.2.1；单头自注意力
K	键矩阵	$n \times d_k$	与查询匹配，形成相关性分数	§3.2.1；单头自注意力
V	值矩阵	$n \times d_v$	提供被加权汇总的内容	§3.2.1；单头自注意力
d_{model}	模型表征维度	base 为 512	各层输入、输出的共同宽度	§3.1
d_k	每头查询和键的维度	base 为 64	控制匹配维度及点积缩放	§3.2.2
d_v	每头值的维度	base 为 64	决定每个头的输出宽度	§3.2.2
h	注意力头数	base 为 8	并行计算多组信息匹配	§3.2.2

一 基础知识讲解

本文讨论的是原始的编码器—解码器 Transformer，任务是机器翻译。先理解下面七个基础概念，再沿架构图和数据形状走一遍。符号表中的 n 表示源序列长度；索引、临时维度和简单运算会在用到时解释。

1.1 序列转导与编码器-解码器框架

概念：**序列转导（sequence transduction）**把一个序列映射为另一个序列，例如英文到德文。**编码器—解码器（encoder-decoder）**将“表示源句”和“生成目标句”分开，编码器产生上下文表征，解码器逐步输出目标词。

机制：解码器同时依据源句表征和已知目标前缀预测下一个词，训练目标能够反向约束两部分如何协同。两部分学到的表示由翻译任务共同约束。

例子：把“我喜欢学习”译为“I like learning”，生成 like 时可以同时利用源句与已生

成的 I。本文保留这一总体框架，改变的是序列内部交换信息的方式。

1.2 RNN/LSTM 的顺序计算瓶颈

概念：**RNN** 的当前隐状态依赖前一步隐状态；**LSTM** 和 **GRU** 用门控改善信息保留，但仍有这条时间依赖。

机制：同一个训练样本内，后一个位置必须等前一个位置完成，限制了沿序列长度方向的并行；不同样本和部分矩阵运算仍能并行。远距离位置的信息还需经过多次状态传递。

例子："那家我上个月和朋友去过、后来又装修了一次的店关门了"，前面的修饰信息需要跨过许多位置传到句尾。原文表 1 将循环层的最长传递路径记为 $O(n)$，而全连接自注意力层为 $O(1)$。这个比较衡量的是信息传递路径；任务中的关系识别效果由后面的实验检验。

1.3 注意力机制（Attention）的基本思想

概念：**注意力（attention）**先用查询与候选键计算匹配分数，再用归一化权重汇总值。Q 问"我要找什么"，K 用来匹配，V 提供内容。

机制：权重随输入变化，且计算可微，翻译损失能训练投影参数，让相关信息在当前任务中更容易被取出。注意力权重不是人工标定的词语重要性。

例子：假设三个候选值获得 0.7、0.2、0.1 的权重，输出就是它们的加权和，第一项贡献最大。本文把这个操作同时用于序列内部的信息交互和源句到译文的对齐。

1.4 自注意力（Self-Attention）

概念：**自注意力（self-attention）**的查询、键、值来自同一条序列，但通常经过不同的可学习投影，三者数值并不相同。

机制：同一层先固定输入表示，再同时计算所有位置间的匹配和汇总。每个位置可以直接读取远处位置，不需要在这一层里逐词递推。

例子："银行旁边有一条河"与"河岸上停着一辆车"中的相关词可以通过不同的上下文权重形成不同表示。本文编码器可读取整个源句；解码器自注意力则受因果掩码限制，只读取已知目标前缀。

1.5 词嵌入与位置信息

概念：**词嵌入（embedding）**把 token 编号映射到向量；**位置编码（positional encoding）**把顺序信息加进向量。原文 base 模型的嵌入宽度是 $d_{model} = 512$。

机制：没有位置编码和位置相关约束时，自注意力对输入的重新排列是等变的，无法仅凭该操作识别原先的绝对词序。位置编码让相同 token 出现在不同位置时具有不同输入表示。

例子："我帮助你"和"你帮助我"包含相同词语，却需要不同输出。本文使用固定

正余弦编码；原文表 3 中可学习位置嵌入也取得接近结果，说明这组实验允许不同的位置表示方案。

1.6 残差连接与层归一化

概念：**残差连接（residual connection）**把输入直接加到子层输出上；**层归一化（LayerNorm）**沿每个位置的特征维度归一化，并带可学习缩放与偏移。

机制：残差提供较直接的信息和梯度通路，归一化控制中间数值尺度，两者共同为深层网络的优化提供支持。

例子：子层只需学习对原表示的增量，而非每次从头重建整个表示。本文采用先残差相加、后归一化的 **Post-LN** 顺序，原文 §3.1 写为：

$$LayerNorm(x + Sublayer(x)) \quad (1)$$

1.7 机器翻译任务与 BLEU 指标

概念：本文在 **WMT 2014** 英德与英法翻译任务上评估，训练数据约为 450 万和 3600 万句对。**BLEU** 根据译文与参考译文的 n-gram 重合情况并结合长度惩罚打分，越高越好。

机制：自动指标使大量译文可以按固定规则比较，但分词、测试集和解码设置会影响结果；同义表达也可能得不到充分奖励。

例子：两种语义相近但用词不同的译文可能得到不同分数，因此要在同一评测协议下解释分差。本文主结果用 newstest2014，消融用 newstest2013，两张表的分数不能混为同一个测试集结果。

二 方法灵感和发表期刊

2.1 发表信息

论文由 **Ashish Vaswani 等 8 位作者**完成，发表于 **NIPS 2017**（会议现称 NeurIPS）。本讲解使用仓库已有的 arXiv:1706.03762v7，首页标注更新日期 2023 年 8 月 2 日；该日期不是论文首次发表年份。以下页码、表号和图号均对应这份 15 页 PDF。

2.2 痛点与动机

原文 §1–2 的出发点是减少序列计算中的顺序依赖。循环模型在一个样本内沿时间步计算；卷积模型虽能同时处理多个位置，但远距离联系通常需要经过多层局部组合。作者希望把远处信息直接接入当前位置，同时保留适合矩阵运算的训练方式。这并不取消自回归生成时逐步产生目标 token 的过程。

2.3 灵感来源

注意力对齐、卷积序列模型、已有的自注意力和记忆网络共同构成了背景。Transformer 的贡献是把它们重组为由多头自注意力、交叉注意力和逐位置前馈网络组成的序列转导架构。标题中的“All You Need”针对替代序列对齐的循环与卷积结构；模型仍使用嵌入、线性层、非线性、归一化和位置编码。

三 方法

3.1 先看 baseline 再看本文改了什么

这里要区分两类 baseline。表 2 的 ConvS2S、GNMT 等是外部实验对照；表 3 的 Transformer base 是消融对照。外部对照用于比较系统效果，内部对照用于分析具体配置。下面选择主论文参考文献 [9] 的 **ConvS2S**，因为它同样追求并行序列建模，核心差别最便于比较。

3.1.1 ConvS2S 的核心与 Transformer 的核心

ConvS2S 在编码器和解码器中堆叠一维卷积，配合门控线性单元与残差连接；局部上下文随层数扩大。它的每个解码层也有读取源句的注意力。Transformer 将序列内部的卷积信息混合改为自注意力，并以多头并行匹配。依据：Gehring 等原论文 §3.2–3.3，链接见文末。

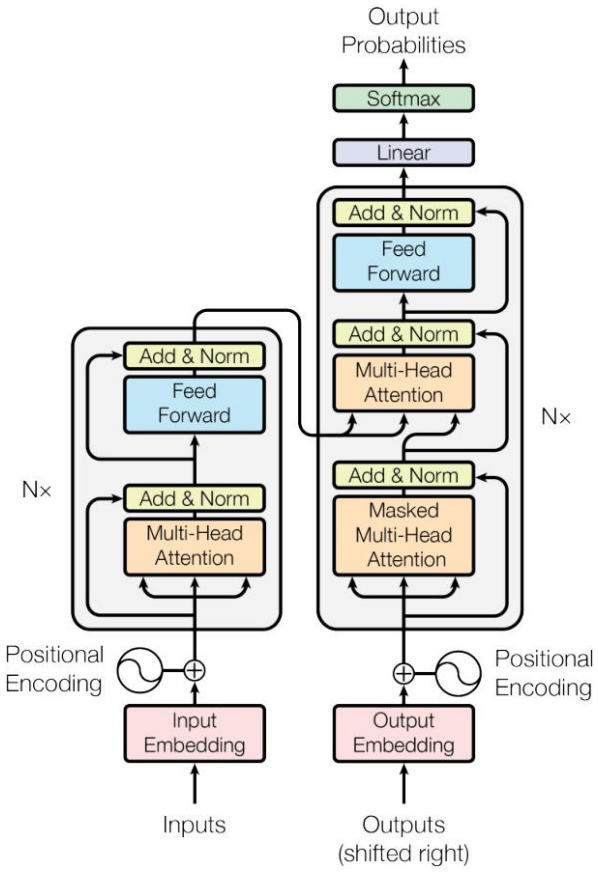
比较点	ConvS2S	本文 Transformer
序列内部信息交换	堆叠局部卷积	自注意力直接连接可见位置
读取源句	各解码层的源端注意力	各解码层的多头交叉注意力
顺序信息	可学习位置嵌入	默认固定正余弦位置编码
生成方式	自回归逐 token 生成	同样自回归逐 token 生成

3.1.2 比较结果时看什么

表 2 同列 WMT 2014/newstest2014 的各模型报告结果，用于比较整套系统。训练数据处理、模型规模和训练预算并不被控制为完全一致，因此分差反映整套系统的差别。要判断某项内部改动的影响，应重点看表 3 在统一开发集设置下的变体实验。

3.2 整体架构：编码器-解码器堆叠

base 编码器和解码器各堆叠 6 层。这里的层数记为 $N = 6$，表示结构重复，各层使用独立参数。编码器每层含多头自注意力和逐位置前馈网络；解码器多一个读取编码器输出的交叉注意力子层。子层外有残差连接和层归一化，外部宽度统一为 $d_{model} = 512$，便于相加。出处：原文 §3.1。



图注：原文 Figure 1（第 3 页）：Transformer 编码器与解码器结构。

从图底部向上读。左侧 Inputs 经嵌入后与位置编码相加；灰框内先做多头自注意力，再经过 Add & Norm、Feed Forward 和 Add & Norm。从下方进入自注意力的三路箭头表示查询、键、值都来自本层输入，但各自投影。绕过子层的箭头是残差支路。

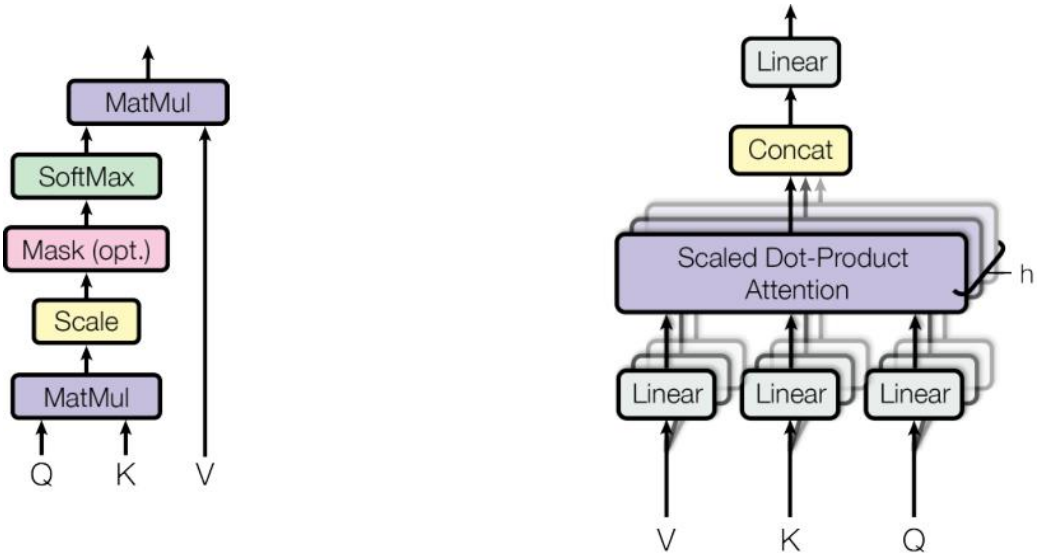
右侧 Outputs (shifted right) 表示训练时目标输入右移一位。第一块 Masked Multi-Head Attention 遮住未来位置；第二块 Multi-Head Attention 的查询来自解码器刚完成的自注意力表示，键和值来自左侧编码器最终输出。随后是前馈网络，顶端 Linear 和 Softmax 把表征转换成目标词表上的预测概率。图中 N× 表示这一结构重复堆叠。

3.3 缩放点积注意力（Scaled Dot-Product Attention）

输入查询、键和值后，缩放点积注意力的计算为：

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad (2)$$

QK^T 计算查询与各个键的匹配分数，T 表示矩阵转置；按行 softmax 后得到权重，再用这些权重汇总 V。原文脚注用一个简化假设解释缩放：若查询和键的各分量独立、均值为 0、方差为 1，点积方差随 d_k 增长，除以 $\sqrt{d_k}$ 可以控制尺度，避免 softmax 过度饱和。缩放解决的是点积分数的尺度问题，优化器、残差与归一化共同影响实际训练。出处：§3.2.1。



图注：原文 Figure 2（第 4 页）：缩放点积注意力与多头注意力。

左图依次是矩阵乘法、Scale 缩放、可选 Mask、Softmax 和与值矩阵的乘法。Mask 要加在 softmax 之前，被遮住的位置分数设为负无穷，归一化后权重为零。右图把这一流程并行执行多次：每个头先独立投影，完成注意力后 Concat 拼接，再做输出线性投影。不同头拥有自己的投影参数，从而形成不同的匹配方式。

3.4 多头注意力（Multi-Head Attention）

多头机制学习多组低维投影，每个头独立匹配并汇总：

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (3)$$

$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O \quad (4)$$

这里 i 是头的编号，W_i^Q, W_i^K, W_i^V 是相应投影，W^O 是拼接后的输出投影。base 取 $h = 8$、$d_k = d_v = 64$；每头从 512 维输入学习一个 64 维视角，8 个输出拼成 512 维。原文说明这一降维设计使总计算成本与全维单头相近；实际开销由投影、张量组织和注意力矩阵计算共同构成。不同头可形成不同关系模式，但其分工不是预先指定的。出处：§3.2.2。

3.5 注意力在模型中的三种用法

三种接法的区别在于信息来源和可见范围。编码器自注意力：查询、键、值来自同一编码器层的输入，可见全部源位置。解码器掩码自注意力：三者来自解码器层输入，但只允许读取当前位置及之前的输入位置；目标序列已右移，因此预测不能看到当前答案。交叉注意力：查询来自本层解码器自注意力之后的表示，键、值来自编码器栈的最终输出。每层解码器都能据此重新读取源句。出处：§3.2.3。

3.6 位置前馈网络（FFN）

注意力负责跨位置交换信息，**FFN** 对每个位置单独进行非线性加工：

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (5)$$

其中 x 为当前位置的行向量，W_1, W_2 是权重，b_1, b_2 是偏置。base 的内部维度 $d_{ff} = 2048$，因此形状经历 512→2048→512；2048 维是这一子层的中间特征宽度。参数在同一层的各位置间共享，不同层用不同参数。宽度升高再压回，使模型能在较大的中间空间中形成非线性特征组合。出处：§3.3。

3.7 嵌入、位置编码与输出层

token 嵌入先乘 $\sqrt{d_{model}}$，再与同宽的位置编码相加。正余弦编码为：

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{model}}\right) \quad (6)$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{model}}\right) \quad (7)$$

pos 是位置，i 在这里是维度对的索引，与多头公式里的头索引不是同一用途。不同频率让位置偏移能以有规律的方式表示。作者选择正余弦的一项理由是它可能支持更长序列；长上下文的适用范围在第 5.5.3–5.5.4 节结合后续研究进一步分析。

解码器顶端的线性层投影到词表，再经 softmax 得到概率。原文 §3.4 描述了两个嵌入层和输出投影的权重共享。英德使用共享的约 3.7 万 BPE 子词词表；英法使用约 3.2 万 word-piece 词表。

3.8 训练策略：优化器与正则化

训练使用源—目标句对。将正确目标序列右移作为解码器输入，预测各位置的正确 token，通过带标签平滑的交叉熵训练全模型；嵌入、注意力投影、FFN、输出层及 LayerNorm 的可学习参数都参与更新，原文采用完整模型的监督训练。固定位置编码和因果掩码没有待学习参数。

Adam 参数为 $\beta_1 = 0.9$、$\beta_2 = 0.98$、$\varepsilon = 10^{-9}$。学习率采用：

$$lrate = d_{model}^{-0.5} \min(s^{-0.5}, s \cdot w^{-1.5}) \quad (8)$$

s 表示训练步数，$w = 4000$ 表示 warmup 步数：先线性增加，再按步数的负二分之一次方下降。base 残差 dropout 为 0.1，标签平滑强度为 0.1。8 张 P100 上，base 训练 10 万步约 12 小时，big 训练 30 万步约 3.5 天。

推理采用束宽 4、长度惩罚 0.6 的 beam search。base 平均最后 5 个检查点，big 平均最后 20 个；这里平均的是同一模型不同训练时刻的参数；表 2 的 Ensemble 则组合多个模型。出处：§5、§6.1。

四 输入内容的变换过程

以下用 **base 配置**跟踪一个批次。设批量大小为 B，源序列长度为 n，目标序列长度为 m；举例时取 $B = 2$、$n = 6$、$m = 5$，只是为了展示形状，不是论文的数据集统

计。训练使用完整目标序列，推理只有逐步增长的目标前缀。

4.1 第一步：从句子到带位置的向量序列

句子分成 token 并转为整数编号，按批次补齐长度，得到 $B \times n$ 的编号表。查嵌入得到 $B \times n \times 512$，乘 $\sqrt{512}$ 后加同位置的正余弦向量，形状不变。训练中嵌入表学习哪些向量有利于预测译文；固定正余弦编码不更新。批处理中的 padding 位置应在注意力计算中被屏蔽，避免把补齐符当成真实词。

4.2 第二步：编码器六层更新上下文表征

4.2.1 自注意力子层的形状变化

每层先从 $B \times n \times 512$ 的输入生成多组投影。按“批次、头、位置、每头维度”排列，Q, K, V 的形状均为 $B \times 8 \times n \times 64$。每头的 QK^T 得到 $n \times n$ 打分，全批次得到 $B \times 8 \times n \times n$。缩放并按键位置归一化后，权重与 V 相乘，输出回到 $B \times 8 \times n \times 64$。拼接 8 头并经过 W^O，得到 $B \times n \times 512$。

在示例中，一头的权重图是 6×6，共有 2×8 张这样的图。行表示哪个位置在提问，列表示它读取哪个位置。学习的是投影参数；注意力权重是输入与当前参数共同计算出的中间结果，不是一张脱离输入、永久固定的表。

4.2.2 FFN 的逐位置特征加工

残差与 LayerNorm 保持 $B \times n \times 512$。接着 FFN 将形状变为 $B \times n \times 2048$，经 ReLU 再投影回 $B \times n \times 512$，然后再次残差和归一化。六层结构重复，各层参数独立。这一过程逐层更新上下文表征；各层形成何种关系模式，可结合注意力可视化与消融来分析。

4.3 第三步：解码器六层，先回忆、再查原文、再加工

训练时，正确目标序列右移后得到 $B \times m$ 编号，再形成 $B \times m \times 512$ 的嵌入。解码器自注意力的权重图为 $B \times 8 \times m \times m$，未来输入位置被遮住；其余位置的权重仍由模型计算。因为输入右移，预测某个目标 token 时，只能看到它之前的目标 token。

交叉注意力的查询长度为 m，键和值的长度为 n，因此权重图为 $B \times 8 \times m \times n$。在示例中，每头是 5×6：5 个目标位置分别查询 6 个源位置。查询来自本层解码器刚处理过的表示，键和值来自编码器最终表示的投影。汇总后恢复 $B \times m \times 512$，再过 FFN、残差与归一化。

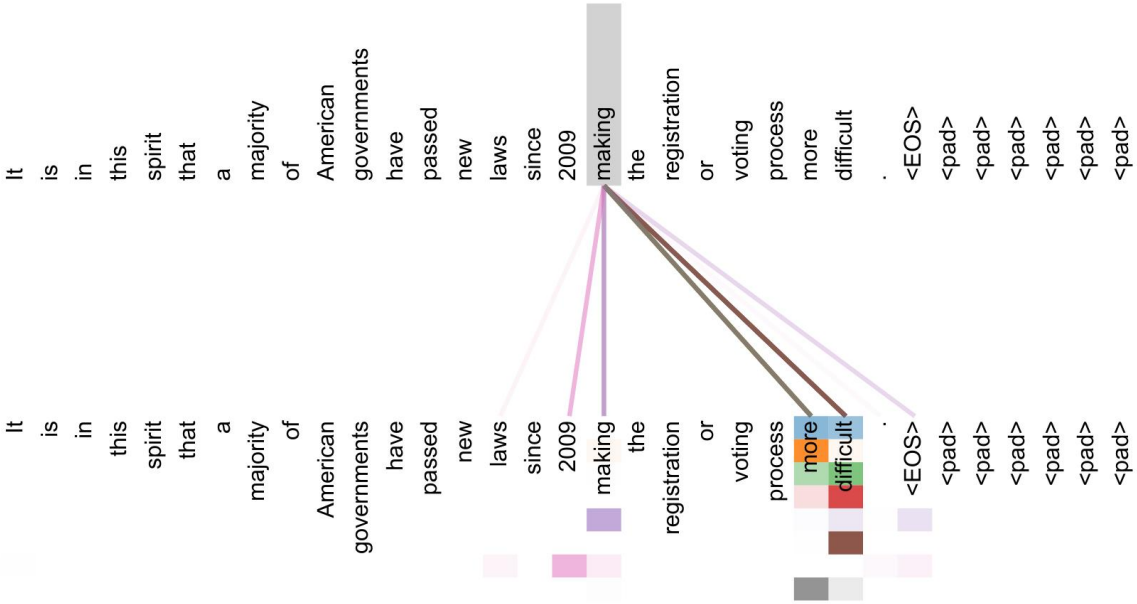
4.4 第四步：从 512 维到下一个词

设词表大小为 U，输出线性层将 $B \times m \times 512$ 变成 $B \times m \times U$ 的 logits。训练时可以同时计算所有目标位置的损失，并通过反向传播更新整个网络。预测概率与正确 token 的差异，会沿输出层、解码器、交叉注意力和编码器影响上游参数。

推理时没有完整正确目标序列：从起始符出发，根据已生成前缀预测下一步，束搜索保留多个候选序列，直到结束符或长度上限。编码器输出可以复用；增量解码还可以缓存历史投影，这样可避免重复计算，目标 token 仍按时间步生成。

4.5 注意力到底学到了什么

原文附录给出了训练后模型的注意力例子。下面这张图只展示编码器第 5 层中 making 这个查询词的权重：不同颜色区分不同头，连线和对应色块表示这些头读取源句各位置的情况。读图时关注这个样本中各个头所连接的位置，以及这些联系如何帮助汇总上下文。



图注：原文 Figure 3（第 13 页）：编码器第 5 层针对 making 的注意力示例。

图中上下两行是同一句话的 token。making 与后面的 more、difficult 之间有明显连线，对应“making … more difficult”的长距离联系；其他头还对 making 本身及别的位置分配权重。这说明自注意力可以在一层中直接读取远处信息。进一步研究头的分工时，可以跨句子统计关系模式，并通过移除某个头观察预测变化。

4.6 把整条数据流连起来

源句 token → 嵌入与位置 → 六层编码器 → 源句上下文表征；目标前缀 → 掩码自注意力 → 交叉注意力读取源句 → FFN → 词表概率。训练时所有预测位置的误差共同更新参数；推理时按目标时间步继续生成。理解 Transformer 的关键，是区分“位置之间如何交换信息”“每个位置如何加工特征”和“目标 token 如何按时间生成”。

五 创新点、可以借鉴的经验

5.1 核心创新与能借鉴的设计

第一，用堆叠自注意力承担序列内部的信息交换，减少循环式逐位置依赖，同时保留编码器—解码器与自回归输出。第二，把多个低维注意力头并行组合，让同一输

入可以形成多种匹配关系。第三，把缩放点积、残差、归一化、FFN 与位置编码组合为可训练的完整系统。这些组件共同构成论文的完整架构。

5.2 主实验结果与原表

表 2 的左侧是模型，BLEU 分成英德 EN-DE、英法 EN-FR 两列，越高越好；训练成本以 FLOPs 估算。先分清单模型、Ensemble 和 Transformer base/big 三组，再沿同一语言方向比较。

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

图注：原文 Table 2（第 8 页）：翻译测试集 BLEU 与估算训练成本。

英德：ConvS2S 为 25.16，最强已列单模型 MoE 为 26.03，ConvS2S Ensemble 为 26.36；Transformer base 为 27.3，big 为 28.4。英法：ConvS2S 为 40.46、其 Ensemble 为 41.29，Transformer big 为 41.8；base 为 38.1，低于这些英法对照。由此可见，优势随模型配置和语言方向变化，big 在两个方向上的结果都更强。

表中 base 训练成本为 3.3×10^{18} FLOPs，big 为 2.3×10^{19}；ConvS2S 英德为 9.6×10^{18}。base 低于后者，big 则高于后者。成本由训练时间、GPU 数和估计吞吐量推算，适合观察系统级成本；相同硬件上的延迟还需另行计时。

数值出处说明：这份 v7 PDF 的摘要和表 2 都写英法 big 为 41.8，而 §6.1 的一处正文写 41.0。本讲解按摘要与原表使用 41.8，并保留这一版本内部差异，便于回查。

5.3 消融实验与 base 对照

表 3 使用英德开发集 newstest2013，base 为 25.8 BLEU、约 6500 万参数。各组改变不同配置，空白项沿用 base。这里没有使用主实验中的检查点平均，解读变体时应与本表的 base 比较，保持数据集和评测设置一致。

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

图注：原文 Table 3（第 9 页）：英德开发集上的结构与训练配置变体。

A 组：1 头为 24.9，8 头 base 为 25.8，16 头为 25.8，32 头为 25.4；改变头数时每头维度也相应调整。B 组：键维度减到 16 时为 25.1，减到 32 时为 25.4。C 组：层数从 6 减到 2 时为 23.7；模型宽度降到 256 时为 24.5，升到 1024 时为 26.0；FFN 宽度升到 4096 时为 26.2，同时参数量上升。

D 组：dropout 从 0.1 改为 0 时为 24.6；标签平滑从 0.1 改为 0 时为 25.3。E 组：可学习位置嵌入为 25.7，与正余弦 base 的 25.8 接近。这组结果给出当前设置下的配置取舍：较大的 FFN 带来收益，位置编码形式的分差较小，移除 dropout 后指标下降明显。

5.4 本文实际解决了什么

原始 Transformer 主要改进的是**有监督序列转导中的信息交换方式与训练并行性**。在能够取得完整源句、拥有成对译文作为训练信号的设置下，它用自注意力替代循环或卷积的序列内部计算，缩短远距离位置的连接路径，并在 WMT 2014 英德、英法任务上取得前述结果。这是一个有效的编码器—解码器架构及其训练方案，原文依据集中在 §3–6、表 1–3。

把它用于实际系统时，需要区分三个层次：架构能够连接哪些信息、训练数据能教会它什么、应用要求它在什么时间和资源约束下输出。原论文重点推进了第一层，并用特定翻译任务验证效果；下面分别考察少量语料、实时输入和长上下文场景还需要补足哪些条件。

5.5 仍未解决的问题与适用场景

检索截至 2026 年 10 月 28 日。 本节核对了 2025–2026 年的低资源翻译、同声传译和长上下文研究，并用 FlashAttention 说明已被后续工作改善的工程限制。文献 [3]–[8] 是后续研究，用于定位今天的应用范围；原始 Transformer 的实验依据仍为文献 [1]。

5.5.1 少量平行语料下如何学好翻译

场景： 为阿卡德语、曼尼普尔语等语种建立翻译系统，可用句对可能只有数万条，训练数据、分词覆盖和可承担的模型规模都与英德、英法大语料实验不同。

证据： Signoroni 等（2025）在约 2.1 万至 5 万句对的数据上研究 Transformer 配置，发现模型容量与翻译质量需要专门权衡，较小配置能显著节约参数并保留较多性能 [3，§3.1、§4.1]。Song 等（2026）的 FLORES-200 评测仍观察到所测小模型在低资源语种上的质量差距，并通过面向低资源语种的蒸馏获得改善 [4，§2.3、§4.3]。回到本文，翻译训练分别使用约 450 万英德句对和 3600 万英法句对 [1，§5.1]；其约 4 万句的小数据实验属于英语句法分析，任务和监督形式不同 [1，§6.3]。

推断： 本文证明了该架构在所测翻译条件下有效，尚未给出“少量句对时怎样取得可靠翻译质量”的完整方案。把它迁移到低资源语种时，需要重新选择分词、容量、正则化以及数据补充或迁移方法，并在目标语种上验证。这里的缺口是低资源翻译的学习与数据条件：原论文尚未验证这一场景，后续配置优化与蒸馏研究已经在部分设置中补足了这个环节。

5.5.2 源句尚未说完时如何及时翻译

场景： 会议字幕或实时口译需要边接收输入边输出，不能总等到整句结束。过早输出可能造成语义或语序错误，等待更多上下文又会增加延迟。

证据： Zhang 等（2026）把同传中的读取/输出决策扩展为分句、删去冗余等动作，在 ACL60/60 英中、英德、英日设置中研究质量与延迟的权衡；其框架仍以文本到文本为核心，联合建模语音输入与动作解码被列为后续工作 [5，§3–4、§6]。原始 Transformer 编码器允许各源位置读取完整源句，解码器通过交叉注意力访问编码器输出；论文给出的推理是自回归束搜索，没有设计何时继续读入、何时立即提交译文的同传策略 [1，§3.2.3、§6.1]。

推断： 本文解决的是已有源序列上的编码与译文生成，训练可并行并不等于具备实时同传能力。用于流式输入时，还需规定源端可见范围、读写时机及延迟目标；面向语音还要处理声学输入。原论文的边界是未覆盖同传决策；后续研究已在增量策略上取得进展，而实时语音系统仍需把输入处理、决策和输出延迟一起评估。

5.5.3 长文本计算如何适配有限显存和时间

场景： 从句子级翻译扩展到长文档或大段代码时，全连接注意力要处理更多位置对，训练和读取上下文的时间、显存开销会成为应用约束。

证据： 本文表 1 给出自注意力的单层计算量 $O(n^2d)$，其中 n 是序列长度、d 是表征宽度；§4 提到可限制注意力范围以处理很长序列，但未在本文中实现这一方案 [1]。FlashAttention 通过分块和减少显存读写保持精确注意力，定理 1 给出的计算量仍为 $O(n^2d)$，而输入输出之外的额外存储随长度线性增长 [6，§3.1，定理 1]。

推断： 原始架构缩短了信息传递路径，却仍需为长序列的两两匹配付出计算代价。后续工作已降低显存存储与读写开销，全连接匹配的计算量仍是这个场景要衡量的因素。真正需要按场景决定的是：在目标长度和硬件预算下，保留全连接关系，还是采用局部、稀疏等改动，并重新测量这些改动对精度、显存和时间的影响。

5.5.4 长上下文中的关键信息如何被可靠使用

场景： 在一整份长文档中查找分散事实、理解多文件代码，或根据长对话中的早期信息回答问题。文本能够输入模型后，还要检验模型能否找到并正确利用相关内容。

证据： NoLiMa（2025）减少问题与证据之间的字面匹配；其表 3 中，所测 GPT-4o 的短上下文基准分数为 99.3，到 32K token 时为 69.7 [7，§4.3–4.4、表 3]。Bansal 等（ICLR 2026）进一步分析长上下文下的注意力分数稀释，提出在测试时更新查询投影的 qTTT，并在所测长上下文任务上得到改善 [8，§2.3、§3、图 4]。原论文的相关证据则是位置间最短连接路径，以及翻译、句法分析任务结果；正余弦编码的长度外推是设计动机之一 [1，§3.5、§4、§6]。

推断： 本文提供了直接连接远处信息的机制，但“直接连接”到“可靠选择和利用证据”之间还有学习与评估问题。长文档应用需要按长度、证据位置和干扰内容测试，并考虑长上下文训练或适配方法。上述后续模型评测把“信息可连接”和“信息能有效使用”分开检验；qTTT 等方法已改善部分设置中的表现。原论文提供的是前者的架构基础，后者仍须结合具体任务、训练方式和评测结果来确认。

5.6 按应用场景选择可借鉴的部分

应用场景	本文可借鉴的部分	落地前需补足或检验
完整源句的监督翻译	编码器—解码器、注意力与 FFN	目标数据、分词和评测协议
少量语料的目标语种	可调整的序列建模架构	容量与数据策略、目标语种质量
实时流式翻译	表征与跨语言对齐模块	源端可见性、读写策略和延迟
长文档或长对话	跨位置交互机制	计算预算、有效检索与证据利用

研究中可先选定一个具体场景，再按“输入条件 → 现有方法缺口 → 本文能提供的模块 → 仍需验证的能力”设计实验。少量语料重点比较数据策略与容量，实时翻译同时报告质量和延迟，长文本同时报告资源消耗与证据利用准确性。这样才能判断复用了 Transformer 的哪一项贡献，以及自己的改进实际补上了哪一处问题。

六 参考文献与定位

[1] Vaswani, A. 等. Attention Is All You Need. NIPS 2017. https://arxiv.org/abs/1706.03762v7。本文图表与页码取自该版本。架构：§3、图 1–2；复杂度：§4、表 1；训练：§5；主实验与消融：§6、表 2–3；注意力可视化：附录图 3。

[2] Gehring, J., Auli, M., Grangier, D., Yarats, D., Dauphin, Y. N. Convolutional Sequence to Sequence Learning. ICML 2017, PMLR 70:1243–1252. https://proceedings.mlr.press/v70/gehring17a.html。对应主论文参考文献 [9]；本文 baseline 机制对比核对其 §3.1–3.3。

[3] Signoroni, E., Rychlý, P., Signoroni, R. Efficient Architectures For Low-Resource Machine Translation. LowResNLP 2025, pp. 39–64. https://aclanthology.org/2025.lowresnlp-1.6/。核对 §3.1、§4.1；低资源数据与配置权衡。

[4] Song, Y. 等. Are Small Language Models the Silver Bullet to Low-Resource Languages Machine Translation? LoResMT 2026, pp. 1–26. https://aclanthology.org/2026.loresmt-1.1/。核对 §2.3、§4.3；多语种评测与蒸馏改进。

[5] Zhang, Q., Yang, Z., Nakamura, S. Redefining Machine Simultaneous Interpretation: From Incremental Translation to Human-Like Strategies. Findings of ACL 2026, pp. 12554–12577. https://aclanthology.org/2026.findings-acl.611/。核对 §3–4、§6；同传动作与适用范围。

[6] Dao, T. 等 . FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. arXiv:2205.14135v2. https://arxiv.org/abs/2205.14135v2。核对 §3.1 定理 1、§3.2；区分计算量与存储开销。

[7] Modarressi, A. 等. NoLiMa: Long-Context Evaluation Beyond Literal Matching. ICML 2025；使用 arXiv:2502.05167v3（2025-07-09）. https://arxiv.org/abs/2502.05167v3。核对 §4.3–4.4、表 3；数值对应该版本与所测模型。

[8] Bansal, R. 等 . Let’s (not) just put things in Context: Test-time Training for Long-context LLMs. ICLR 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/b619cd6dcc986856b8a8da2b08d89396-Abstract-Conference.html。核对 §2.3、§3、图 4；测试时查询投影更新与长上下文利用。